خطأ برمجي كلف الملايين

مسبار المريخ Mars Climate Orbiter (1999): رقم بدون وحدة

الاسم: تميم تركي الحارثي

الكلاس: 103

البحث: بحث البايثون – الموضوع 1

التاريخ: 1 نوفمبر 2026

المقدمة

هذا البحث عن مسبار ناسا «مارس كلايمت أوربتر» اللي ضاع عند المريخ سنة 1999 بسبب خطأ بسيط جدا: فريق أرسل أرقام بوحدة، والفريق الثاني قراها كأنها بوحدة ثانية. اخترت هالموضوع لأنه يوضح إن أشياء ندرسها ونحسبها سهلة، مثل الوحدات وتحويل نوع البيانات، ممكن تسبب خسارة ملايين الدولارات لو أحد ما انتبه لها، ولأنه قريب من اللي سويناه بالبايثون لما كتبنا «SAR» جنب كل مبلغ.

المحتوى

وش صار؟

انطلق المسبار في ديسمبر 1998 عشان يدرس جو المريخ ومناخه. وفي 23 سبتمبر 1999، لما وصل المريخ وبدأ يدخل مداره، انقطع الاتصال فيه وما رجع. لجنة التحقيق في ناسا لقت إن المسبار كان أقرب بكثير للمريخ من المخطط: المفروض يمر على ارتفاع تقريبا 226 كم، بس الحسابات بعدين بينت إنه مر على ارتفاع حوالي 57 كم بس، وهذا قريب لدرجة إن المسبار غالبا احترق بالغلاف الجوي أو انحرف ولف حول الشمس. وكانت تكلفة المسبار نفسه حوالي 125 مليون دولار، غير سنين من شغل العلماء.

ليش صار؟

برنامج على الأرض كتبته شركة لوكهيد مارتن كان يحسب قوة دفع المحركات الصغيرة اللي تعدل اتجاه المسبار، وكان يطلع الناتج بوحدة إنجليزية (رطل-قوة × ثانية). وبرنامج الملاحة في مختبر JPL التابع لناسا كان يستقبل هالأرقام ويفترض إنها بالوحدة المترية (نيوتن × ثانية)، مثل ما هو مكتوب في اتفاق الواجهة بين الفريقين. الرقم نفسه ما فيه أي علامة توضح وحدته، فما أحد حس بالمشكلة. والفرق بين الوحدتين تقريبا 4.45 مرة، فكل تعديل صغير بالمسار كان يتحسب أصغر من حقيقته، وتراكم الخطأ شهور لين صار المسار غلط.

كيف كان الاختبار بيكتشفه؟

حسب تقرير اللجنة، الملف اللي فيه الأرقام ما انختبر صح بين البرنامجين، يعني كل فريق اختبر برنامجه لحاله، بس ما أحد اختبر **الاثنين مع بعض**. لو سووا اختبار بسيط: يدخلون قيمة معروفة (مثلا 10) ويتأكدون إن اللي يوصل لبرنامج الملاحة 44.48 مو 10، كان انكشف الخطأ. وكمان الفريق لاحظ وقتها إن المسار فيه شي غريب، بس ما وقفوا يتحققون ويبلغون بشكل رسمي. ونفس الفكرة صارت مع صاروخ **Ariane 5** سنة 1996: برنامج حاول يحول رقم كبير من نوع عشري إلى عدد صحيح صغير (16 بت) ما يتسع له، فطلع خطأ وانفجر الصاروخ بعد حوالي 40 ثانية من الإطلاق، لأنهم ما اختبروا الكود القديم مع أرقام الصاروخ الجديد.

ربطه بالدروس

المفهوم (الوحدة 2)	وين ظهر بالموضوع
أنواع البيانات والتحويل	الرقم بالكمبيوتر مجرد قيمة، وما يعرف هو رطل ولا نيوتن. بالبايثون `input()` يرجع نص، وإحنا لازم نحوله بـ `float()` أو `int()` بشكل صريح. بالمسبار كان لازم تحويل صريح بين الوحدتين (× 4.45) وما صار. وبـ Ariane 5 صار تحويل نوع (من عشري لعدد صحيح صغير) بدون ما يتأكدون إن الرقم يتسع، مثل لما نسوي `int("three")` ويطلع `ValueError`.
الاختبار واصطياد الأخطاء	درسنا إن الأخطاء ثلاث أنواع: Syntax و Runtime و Logic. خطأ المسبار كان **خطأ منطقي (Logic)**: البرنامج اشتغل عادي بدون أي رسالة خطأ، بس النتيجة غلط. وهذا أخطر نوع لأنه ما يبان إلا بالاختبار بحالات متوقعة (مثل لما قارنا ناتج الفاتورة بـ 13.80). أما Ariane 5 فكان **خطأ وقت تشغيل (Runtime)** وقف النظام.
المخرجات بـ f-string	تعودنا نكتب الوحدة جنب كل رقم (مثل `{total:.2f} SAR`). لو كانت ملفات المسبار تكتب الوحدة جنب الرقم، كان الفريق الثاني انتبه.

مثال بايثون

برنامج صغير يحول قوة الدفع من «رطل-قوة × ثانية» إلى «نيوتن × ثانية»، ويوضح كم كان الخطأ لو استخدمنا الرقم بدون تحويل (مثل اللي صار بالمسبار). فيه `input()` مع تحويل `float()`، وتعبير حسابي، ومخرجات بـ f-string فيها الوحدة، وتعليقات.

```
# Impulse unit converter - inspired by the Mars Climate Orbiter mishap (1999)
# One team gave impulse in pound-force seconds (lbf*s),
# the other team expected newton seconds (N*s). Nobody converted.

LBF_S_TO_N_S = 4.44822  # 1 lbf*s equals 4.44822 N*s

# Input: read the value as text, then convert it to a number (float)
impulse_text = input("Thruster impulse in pound-force seconds (lbf*s), e.g. 10:
")
impulse_lbf_s = float(impulse_text)

# Processing: the correct conversion (arithmetic expression)
impulse_n_s = impulse_lbf_s * LBF_S_TO_N_S

# The mistake: using the number as if it were already in N*s
wrong_value_n_s = impulse_lbf_s
error_ratio = impulse_n_s / wrong_value_n_s

# Output: always show the unit next to every number
print(f"Value entered:          {impulse_lbf_s:.2f} lbf*s")
print(f"Correct value:          {impulse_n_s:.2f} N*s")
print(f"Value used by mistake:  {wrong_value_n_s:.2f} N*s")
print(f"The real effect was {error_ratio:.2f} times bigger than the software
thought.")
```

شرح الكود: أول شي عرفت ثابت التحويل `LBF_S_TO_N_S`. بعدين أخذت الرقم من المستخدم كنص وحولته لـ `float`. ضربته بثابت التحويل عشان يطلع الرقم الصحيح بالنيوتن. بعدين حسبت «القيمة الغلط» اللي هي نفس الرقم بدون تحويل، وقسمت الصح على الغلط عشان يطلع كم مرة كان الفرق. وآخر شي طبعت كل رقم ومعه وحدته.

صورة التشغيل:

```
Thruster impulse in pound-force seconds (lbf*s), e.g. 10: 10
Value entered:          10.00 lbf*s
Correct value:          44.48 N*s
Value used by mistake:  10.00 N*s
The real effect was 4.45 times bigger than the software thought.
```

الخاتمة

شي تعلمته: إن الرقم بدون وحدة أو بدون نوع واضح ممكن يكون أخطر من خطأ يوقف البرنامج، لأنه يمشي بهدوء ويطلع نتيجة غلط. وإن الاختبار لازم يكون للنظام كامل مع بعض، مو كل جزء لحاله بس.

سؤال بقى عندي: هل فيه لغات برمجة أو مكتبات بايثون تخلي الرقم يحمل وحدته معه، بحيث البرنامج نفسه يرفض تجمع رطل مع نيوتن؟

المصادر

1. NASA – Mars Climate Orbiter Mishap Investigation Board, Phase I Report (1999):
   https://llis.nasa.gov/llis_lib/pdf/1009464main1_0641-mr.pdf
2. NASA Jet Propulsion Laboratory – Mars Climate Orbiter mission page:
   https://www.jpl.nasa.gov/missions/mars-climate-orbiter
3. ESA – Ariane 5 Flight 501 Failure, Report by the Inquiry Board (1996):
   https://esamultimedia.esa.int/docs/esa-x-1819eng.pdf
4. Python Documentation – Built-in Functions (input, int, float):
   https://docs.python.org/3/library/functions.html

> **استخدام أدوات الذكاء الاصطناعي:** _____ (اكتب هنا وش استخدمتها فيه، مثلا: فهم تقرير ناسا وترتيب الأفكار ومراجعة الكود).